

DATWAVE × BIP

HACKATHON

Agentic AI

Trading Agent on a Simulated Market

Two days · Teams of 3–4 people · Alpaca Paper Trading

100K USD Virtual Portfolio

Real Delayed Market Data

No Real Money

MISSION

Build the most intelligent agent.



PERCEIVE

Read live prices & news via tool calls — never from memory



REASON

Produce explicit written rationale before every decision



ACT

Execute orders on the simulated broker autonomously



LEARN

Maintain a trade journal with timestamp, ticker & outcome

Choose your ambition level at kick-off.

1

Core Agent

Minimum bar — everyone

- Retrieve prices & news via tool calls (no hallucinated data)
- Produce written rationale for each decision
- Record every decision in a structured trade journal
- No order execution required

2

Executing Agent

Intermediate

- *Everything in Level 1, plus:*
- Execute real orders on simulated broker
- Update portfolio state after each trade
- Handle at least one tool failure gracefully

3

Autonomous Agent

Advanced

- *Everything in Level 2, plus:*
- Run autonomous loop for 30+ minutes
- Memory / context layer across decision cycles
- Non-trivial risk logic (sizing, stop-loss, diversification)

Build fast, think agentic.

09:00–09:30

Kick-off & Team Formation

Level declaration · Starter kit · API keys

09:30–10:00

Tech Talk: Anatomy of a Trading Agent

Perception→Reasoning→Action loop · Tool use · Alpaca live demo

10:00–13:00

Sprint 1 — Working Prototype

End-to-end cycle: fetch → decide → record · Checkpoint at 12:00

13:00–14:00

Lunch + Informal Demo

2-min status per team · Technical feedback only

 **First Trade**

14:00–17:30

Sprint 2 — Tools & Memory

News search · Trade journal · Error handling · Optional memory layer

 **Best Reasoning @ 16:00**

17:30–18:00

Daily Stand-up (attendance required)

3 min per team · Day 1 mini-challenge results

Ship it, present it.

09:00–09:20

Morning Sync

Final priorities · Demo format reminder (5+3 min) · Code freeze @ 15:00

09:20–13:00

Sprint 3 — Polish & Stress Test

Robustness · Repeatable demo scenario · Stability only — no new features

⚡ Best Recovery @ 11:00

13:00–15:00

Lunch & Final Debugging

Max 3 context slides · Demo script preparation

⚡ Best Journal @ 13:00

15:00–16:30

Final Demo

5 min live demo + 3 min Q&A · Agent runs autonomously · Trade journal visible

16:30–17:00

Deliberation & Awards

Main category winners + Special prizes announced

17:00–18:00

Open Retrospective

What did you learn? Technical discussion · Feedback for future editions

EVALUATION

The jury does not evaluate profit.

Agentic behaviour is the object of evaluation.

40

**Demonstrable
Functionality**

pts

Agent completes real end-to-end operations without crashing. Handles at least one tool error gracefully. Evaluated relative to declared level.

35

**Reasoning
Quality**

pts

Decisions are well-motivated, traceable in the journal, consistent with received data. Agent never invents prices. Knows when to stop.

25

**Originality
of Use Case**

pts

Strategy or architecture shows something non-obvious: unusual tool, risk-management logic, or effective memory layer.



Special Prize: Best Failure Recovery

Awarded to the team whose agent handles an unexpected error most intelligently during the demo. Does not affect the main score.

MINI-CHALLENGES

Keep the energy high. No score impact — symbolic recognition only.

Day 1 · Lunch

First Trade

First team to execute a real order with documented reasoning

 **First Mover**

Day 1 · 16:00

Best Reasoning

Clearest and most well-motivated reasoning step from any trade journal

 **Best Analyst**

Day 2 · 11:00

Best Recovery

Most elegant handling of a tool error injected silently by mentors

 **Anti-Fragile**

Day 2 · 13:00

Best Journal

Most readable and informative trade journal: structure, detail, usefulness

 **Best Logger**

Complete these steps the day before.

1 — Account & API Keys

- Create Alpaca Markets account → activate Paper Trading
- Save API Key & Secret in .env (never in the repo)
- Optional: Twelve Data or Polygon.io for extra market data

2 — Dev Environment

- Python 3.13+ installed
- Create virtualenv with uv
- Install: alpaca-py, openai or anthropic, python-dotenv, requests
- Run `python test_connection.py` — must pass

3 — Starter Repo

- Fork the repo provided by organizers
- Read README and ARCHITECTURE.md fully
- Make at least one test call to Alpaca (get account balance)

4 — Team Formation

- Agree on primary laptop
- Declare ambition level (1 / 2 / 3)
- Choose a team name and communicate to organizers

TECH STACK

Recommended — not mandatory. The starter repo is built around this.

-  **Agent Framework** LangGraph (recommended) or LangChain AgentExecutor
-  **Simulated Broker** Alpaca Paper Trading — REST + WebSocket, free tier
-  **Market Data** Alpaca Market Data · yfinance (historical) · Twelve Data
-  **News** NewsAPI.org (100 req/day free) · Alpaca News Feed
-  **LLM** GPT-4o · Claude Sonnet · Gemini Flash · Ollama (offline)
-  **Journaling** Local SQLite or structured JSON — simple and inspectable

The final demo.



Live demo only — no pre-recorded video



Agent runs autonomously for 5+ consecutive minutes, visible to the jury



Trade journal must be displayable during the presentation



5 minutes live + 3 minutes Q&A per team



Presentation order drawn by lot on the morning of Day 2



Max 3 context slides — optional

Now go build the most intelligent agent. Good luck. 🚀

09:30 – 10:00

Tech Talk

Anatomy of a Trading Agent

1. Perception → Reasoning → Action loop

2. Tool use: get_price · search_news · place_order

3. Context management & hallucination prevention

4. Alpaca Paper Trading: live API demo

What makes it an agent, not a chatbot.

PERCEIVE

Observe the world

- Live price feed via `get_price()`
- News headlines via `search_news()`
- Portfolio state from broker API
- Never from the model's own memory



REASON

Think before acting

- Analyse all retrieved data
- Weigh risk vs. opportunity
- Write explicit rationale in natural language
- Decide: buy / sell / hold / wait



ACT

Change the world

- Call `place_order()` on the broker
- Log every decision to the trade journal
- Update internal portfolio state
- Handle errors — then loop again

↻ Loop repeats autonomously — no human input between cycles

Three tools. That's your agent's entire interface with the world.

`get_price(ticker)`



Returns: { ticker, price, timestamp, volume }

- Always verify timestamp — stale data is worse than no data
- If the tool returns an error: retry once, then log 'data unavailable' and skip
- Never use a price the model 'remembers' from training

```
{ "ticker": "AAPL",  
  "price": 189.42,  
  "timestamp": "2024-06-15T14:32:01Z",  
  "volume": 1240300 }
```

`search_news(query)`



Returns: [{ headline, source, published_at, summary }]

- Use the ticker as query: 'AAPL earnings' not 'stock market news'
- Summarise — don't paste raw headlines into the prompt context
- Ignore articles older than the current trading day

```
[ { "headline": "Apple beats Q2",  
    "source": "Reuters",  
    "published_at": "2024-06-15",  
    "summary": "EPS $1.53 vs $1.50 est." } ]
```

`place_order(ticker, side, qty)`



Returns: { order_id, status, filled_price } or error

- Always check portfolio balance before placing — never over-invest
- side: 'buy' or 'sell' | qty: number of shares (integer)
- On rejection: log the error, reason, and do NOT retry blindly

```
{ "order_id": "a1b2c3",  
  "status": "filled",  
  "filled_price": 189.50,  
  "filled_qty": 10 }
```

How to prevent the LLM from inventing data.

⚠ The Hallucination Trap

Prompt says: "What is the price of AAPL?"

LLM answers: "AAPL is currently trading at \$187.20"

Reality: *Model has no real-time data.*

Result: *Agent places order based on a made-up price.*

Rule 1 violation: Every data point the agent uses to make a decision **MUST** come from a documentable tool call. Not from what the model was trained on.

✓ The Grounded Prompt

```
# Market data (from tool call)
AAPL price: $189.42 @ 14:32 UTC
Volume: 1,240,300 (above 30d avg)

# News (from tool call)
Apple beats Q2 EPS by 2% (Reuters)

# Portfolio state
Cash: $82,450 | AAPL: 0 shares

Based ONLY on the above data,
decide: buy / sell / hold.
```

Your simulated broker — live demo.

1 — Account State

GET /v2/account

```
{ "portfolio_value":  
  "100142.50",  
  "cash": "82450.00",  
  "buying_power": "82450.00",  
  "equity": "100142.50" }
```

- Always check buying_power before placing
- portfolio_value updates in real time

2 — Place Order

POST /v2/orders

```
{ "symbol": "AAPL",  
  "qty": "10",  
  "side": "buy",  
  "type": "market",  
  "time_in_force": "day" }
```

- type: 'market' fills immediately
- time_in_force: 'day' cancels at close

3 — Open Positions

GET /v2/positions

```
[ { "symbol": "AAPL",  
    "qty": "10",  
    "avg_entry_price":  
      "189.50",  
    "current_price":  
      "191.20",  
    "unrealized_pl": "17.00"  
  } ]
```

- unrealized_pl is your live P&L
- Use this to track open risk

```
from alpaca.trading.client import TradingClient | client = TradingClient(API_KEY, SECRET_KEY,  
paper=True)
```

SYSTEM PROMPT BLUEPRINT

The minimal structure that ties it all together.

ROLE

You are an autonomous trading agent operating on a simulated stock market.
Your goal is to make well-reasoned buy/sell/hold decisions.

TOOL OUTPUTS

```
# Prices (from get_price): {prices}
# News (from search_news): {news}
# Portfolio (from get_account): {portfolio}
```

CONSTRAINTS

NEVER use prices or news from your training data.
ALL market data MUST come from tool calls. If a tool fails, log the failure and do not act.

OUTPUT FORMAT

Respond in this exact format:
RATIONALE: <explain your reasoning>
DECISION: <BUY|SELL|HOLD>
ACTION: <call place_order or skip>

Your agent's memory — and the jury's evidence.

Required fields (Level 1+)

```
timestamp:      "2024-06-15T14:32:05Z"

ticker:         "AAPL"

decision:       "BUY"  // or SELL / HOLD
                "EPS beat + price dip
                below
rationale:      20d MA — asymmetric
                risk"

data_source:    "get_price @ 14:32 UTC"

outcome:        "filled @ 189.50, qty:
                10"
```

What the jury looks for

- ✓ Rationale cites the actual data received, not generic reasoning
- ✓ data_source is traceable — timestamp proves it came from a tool
- ✓ HOLD decisions are logged too — inaction is a decision
- ✓ Errors are logged with error_code, not silently skipped
- ✗ *"I decided to buy because AAPL is a strong company" — not grounded*
- ✗ *Missing rationale field — this alone fails the reasoning criterion*